

Infosys - 100 Technical Interview Questions

Infosys Limited

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. What happens when a class implements an interface but does not implement all its methods?

Answer:

If a class implements an interface but doesn't implement all its abstract methods, the class itself must be declared abstract; otherwise, it is a compile-time error since an interface is a contract that guarantees every method has an implementation somewhere in the concrete class hierarchy.

Q2. What is an object in OOP, and how is it different from a class?

Answer:

An object is a concrete instance of a class, with its own state (field values) stored in memory, while a class is the blueprint/template that defines what fields and methods its objects will have. 'Car' is the class; 'myRedCar' is an object of that class.

Q3. Explain the difference between an abstract class and an interface. When would you choose one over the other?

Answer:

An abstract class can mix abstract and concrete methods, hold state (fields), and a class can extend only one abstract class. An interface (pre-Java 8) only declares method signatures with no implementation, and a class can implement many interfaces. Use an abstract class when subclasses share common state/behaviour; use an interface when you only need to guarantee a contract across unrelated classes.

Q4. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

Q5. What is constructor overloading? Can constructors be inherited?

Answer:

Constructor overloading means a class has multiple constructors with different parameter lists, letting objects be created in different ways (e.g. `Point()` vs `Point(x,y)`). Constructors are NOT inherited by subclasses - a subclass must define its own constructors, though it can call a parent constructor explicitly via `super()`.

Q6. What is a functional interface in Java, and how does it relate to lambda expressions?

Answer:

A functional interface is an interface with exactly one abstract method (it may have default/static methods too), e.g. Runnable or Comparator. Java's lambda expressions are essentially shorthand implementations of a functional interface's single abstract method, letting you write (a, b) -> a.compareTo(b) instead of an anonymous inner class.

Q7. Describe a scenario from your own project where you used inheritance, and explain why it was the right choice.

Answer:

Example answer structure: describe a project where a base class captured shared behaviour (e.g. a base Notification class with a send() method) and subclasses (EmailNotification, SMSNotification) overrode send() with channel-specific logic - explain why this avoided duplicating shared validation/logging code and made adding a new channel a matter of adding one new subclass.

Q8. Explain the difference between 'is-a' and 'has-a' relationships with examples.

Answer:

An 'is-a' relationship implies inheritance - a Dog is-a Animal. A 'has-a' relationship implies composition - a Car has-a Engine. Choosing 'has-a' over 'is-a' when there isn't a true specialization relationship avoids incorrectly forcing an inheritance hierarchy where composition would be more appropriate.

Q9. What design pattern would you use to ensure only a single instance of a class exists, and how would you implement it thread-safely?

Answer:

The Singleton design pattern ensures only one instance of a class exists. A thread-safe implementation typically uses either double-checked locking with a volatile instance field, an eagerly-initialized static final instance, or (in Java) a single-element enum, all of which avoid two threads creating two separate instances under concurrent access.

Q10. Explain the Open/Closed Principle with a practical example.

Answer:

The Open/Closed Principle says classes should be open for extension but closed for modification. Practical example: instead of adding an if-else chain to a PaymentProcessor class every time a new payment method is added, define a PaymentMethod interface and add new payment types as new classes implementing it - existing code doesn't need to change.

Q11. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. add(int,int) and add(double,double). Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. Dog overriding Animal's makeSound().

Q12. What is operator overloading? Give an example in C++.

Answer:

Operator overloading lets you redefine what an operator like '+' does for a user-defined type. For example, in C++ you could overload '+' for a Complex number class so that `c1 + c2` adds their real and imaginary parts, letting Complex objects be used with familiar arithmetic syntax.

Q13. What is method hiding, and how is it different from method overriding?

Answer:

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its parent - the parent's version isn't overridden, just hidden, and which one runs depends on the reference type used to call it (resolved at compile time). Method overriding, in contrast, applies to instance methods and is resolved at runtime based on the actual object type.

Q14. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q15. Explain the concept of late binding vs early binding with an example.

Answer:

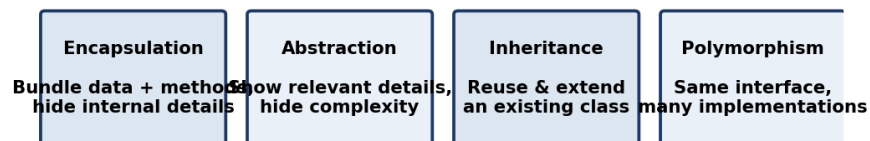
Early (static) binding resolves which method to call at compile time, based on the reference type - used for overloaded and static methods. Late (dynamic) binding resolves the method call at runtime, based on the actual object type - used for overridden instance methods, which is the basis of runtime polymorphism.

Q16. Explain the four pillars of Object-Oriented Programming with a real-world example for each.

Answer:

The four pillars are: **Encapsulation** - bundling data and methods together and hiding internal state (e.g. a bank account class exposing `deposit()/withdraw()` but hiding the raw balance field); **Abstraction** - exposing only relevant details, like a car's steering wheel hiding the mechanics underneath; **Inheritance** - a Manager class reusing and extending an Employee class; **Polymorphism** - a `draw()` method behaving differently for a Circle vs a Square object called through a common Shape reference.

Four Pillars of OOP



DBMS & SQL

Q17. What is database replication, and what is the difference between synchronous and asynchronous replication?

Answer:

Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Q18. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q19. What is a view in SQL, and how is it different from a materialized view?

Answer:

A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q20. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q21. What is sharding, and how does it help scale a database horizontally?

Answer:

Sharding splits a large table's rows horizontally across multiple database servers (shards), typically based on a shard key (e.g. customer_id range or hash), so no single machine needs to store or query the entire dataset. It improves write and storage scalability but introduces challenges like cross-shard joins/transactions and rebalancing when adding new shards.

Q22. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Q23. Explain the difference between horizontal and vertical partitioning of a table.

Answer:

Horizontal partitioning splits a table by rows (e.g. orders from 2023 in one partition, 2024 in another), similar in spirit to sharding but often within a single database instance. Vertical partitioning splits a table by columns (e.g. splitting rarely used large text/blob columns into a separate table), reducing the size of rows scanned for common queries that don't need those columns.

Q24. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q25. What is indexing, and what are the trade-offs of adding too many indexes to a table?

Answer:

An index is a data structure (commonly a B-Tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT queries. The trade-off is that every INSERT/UPDATE/DELETE must also update all indexes on that table, slowing down writes and consuming extra storage - so indexes should be added selectively on columns that are frequently filtered or joined on.

Q26. Explain the difference between a correlated subquery and a regular subquery.

Answer:

A regular (uncorrelated) subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query and is conceptually re-evaluated for each row processed by the outer query, e.g. finding employees who earn more than the average salary of their own department.

Q27. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q28. How does a B-Tree index improve query performance internally?

Answer:

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically $O(\log n)$) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q29. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department). The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q30. Write a SQL query to find the Nth highest salary using a window function like RANK() or DENSE_RANK().

Answer:

SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM Employee) ranked WHERE rn = N (substituting the desired N). DENSE_RANK() assigns ranks without gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.

Q31. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

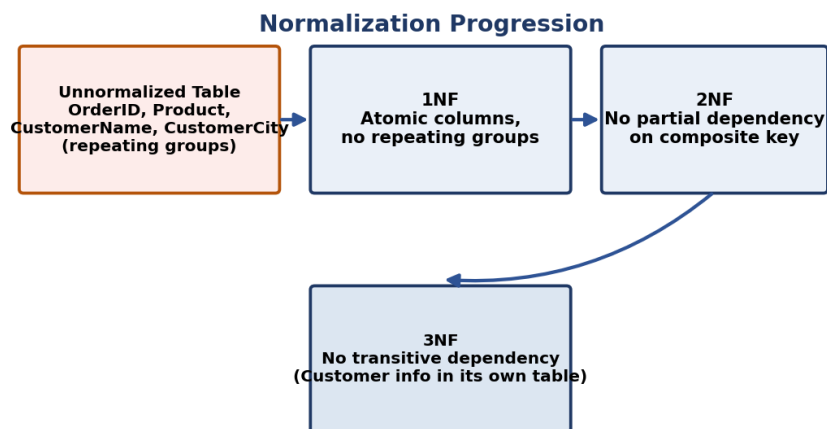
Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q32. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with a practical example.

Answer:

Normalization removes redundancy step by step: **1NF** requires atomic column values with no repeating groups; **2NF** requires 1NF plus no partial dependency of a non-key column on part of a composite primary key; **3NF** requires 2NF plus no transitive dependency (a non-key column depending on another non-key column); **BCNF** is a stricter version of 3NF handling certain anomalies where a table has multiple overlapping candidate keys. Example: splitting an Orders table that repeats CustomerCity for every order into a separate Customers table removes the transitive dependency.



Q33. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q34. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Operating Systems

Q35. What is a semaphore? Differentiate between binary and counting semaphores.

Answer:

A semaphore is an integer variable used for process synchronization, with atomic increment (signal/V) and decrement (wait/P) operations. A binary semaphore takes only values 0 or 1 and behaves like a mutex, ensuring exclusive access. A counting semaphore can take any non-negative integer, used to manage a pool of multiple identical resources (e.g. a fixed number of database connections).

Q36. How does an operating system ensure fair CPU allocation among multiple processes?

Answer:

The OS ensures fair CPU allocation through its scheduler, using algorithms like Round Robin (equal time slices for each process) or priority-based scheduling with aging (gradually increasing the priority of processes that have waited a long time, to prevent starvation), combined with preemption so no single process can monopolize the CPU indefinitely.

Q37. What is the difference between multiprogramming, multitasking, and multiprocessing?

Answer:

Multiprogramming keeps multiple programs in memory so the CPU always has something to execute when one process is waiting on I/O, improving utilization on a single CPU. Multitasking extends this idea to give the illusion of several programs running 'simultaneously' via time-slicing, often in an interactive context. Multiprocessing refers to a system with multiple physical CPUs/cores actually executing instructions in parallel at the same instant.

Q38. Explain how a page fault is handled by the operating system.

Answer:

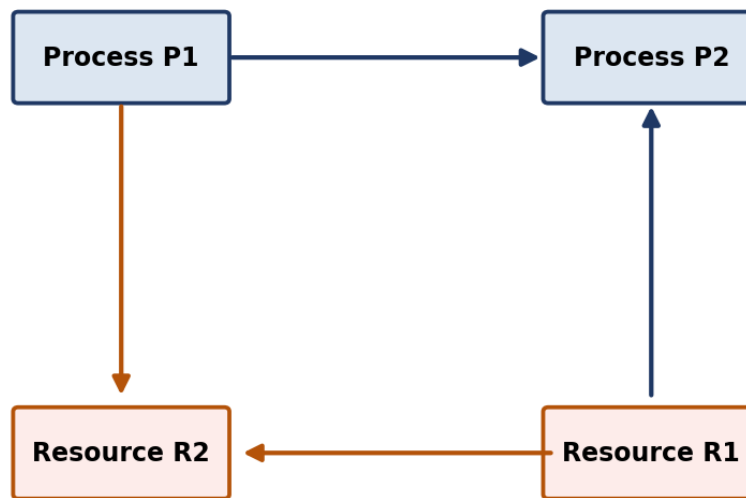
When a process accesses a page not currently in physical memory, the CPU raises a page fault trap. The OS then: locates the required page on disk, finds a free frame (or selects a victim page to evict via a page replacement algorithm like LRU, writing it back to disk if modified), loads the required page into that frame, updates the page table, and resumes the instruction that caused the fault.

Q39. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q40. Explain what an inode is and what information it stores.

Answer:

An inode is a data structure in Unix-like file systems that stores a file's metadata: permissions, owner, size, timestamps, and pointers to the disk blocks holding the actual file data - notably, it does not store the file's name (that mapping lives in the containing directory), which is why hard links to the same inode can have different names.

Q41. Describe a real-world analogy to explain deadlock to a non-technical person.

Answer:

Analogy: imagine four cars at a four-way intersection, each waiting for the car to its right to move first before it can proceed - if all four wait simultaneously, none will ever move, even though each car is only blocked because of the others' identical behavior. That's a deadlock: everyone is waiting for someone else to act first, and no one ever will.

Q42. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q43. What is the difference between internal and external fragmentation?

Answer:

Internal fragmentation is wasted space inside an allocated fixed-size block that's larger than what the process actually needs (common in paging). External fragmentation is wasted space between allocated blocks - free memory exists in total but is scattered in pieces too small individually to satisfy a new request (common in segmentation/variable-size allocation).

Q44. Explain how the Least Recently Used (LRU) page replacement algorithm works, and how you might implement it efficiently.

Answer:

LRU (Least Recently Used) evicts the page that hasn't been used for the longest time when a new page must be loaded and memory is full, based on the assumption that recently used pages are likely to be used again soon. An efficient implementation uses a combination of a hashmap (for $O(1)$ lookup of a page's location) and a doubly linked list ordered by recency, so both finding a page and moving it to the 'most recently used' end take $O(1)$ time.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q45. What is the difference between a hard link and a symbolic link in a file system?

Answer:

A hard link is essentially another directory entry pointing directly to the same inode (and underlying data) as the original file - deleting the original name still leaves the data accessible via the hard link, and it cannot span file systems. A symbolic (soft) link is a small separate file that stores the path to the target file - it can point across file systems or to directories, but becomes a 'dangling link' if the target is deleted or moved.

Q46. What is spooling, and how does it help manage I/O devices like printers?

Answer:

Spooling (Simultaneous Peripheral Operations On-Line) buffers data intended for a slow I/O device (like a printer) on disk, letting a process 'print' instantly to the spool without waiting for the physical device, while a separate spooler process feeds jobs to the device at its own pace - allowing multiple processes to share one device without conflict.

Computer Networks

Q47. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

Q48. Explain how SSL/TLS certificates establish trust between a client and a server.

Answer:

An SSL/TLS certificate, issued by a trusted Certificate Authority (CA), binds a public key to a domain's identity. During the TLS handshake, the server presents its certificate; the client verifies it was signed by a CA it trusts (via the certificate chain up to a trusted root CA), confirming the server is who it claims to be before establishing an encrypted session using keys derived during the handshake.

Q49. What is the difference between a stateful and stateless protocol?

Answer:

A stateful protocol (like FTP or a typical database connection) requires the server to remember information about the client's session across multiple requests. A stateless protocol (like HTTP by default) treats every request independently, with no memory of previous requests - any state needed (like login status) must be carried explicitly in each request, e.g. via cookies or tokens.

Q50. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q51. What is a CDN, and how does it reduce latency for end users?

Answer:

A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Q52. What is the difference between TCP and UDP? Give an example use case for each.

Answer:

TCP is connection-oriented, establishes a session via a handshake, guarantees reliable and ordered delivery through acknowledgments and retransmission, but has more overhead - used for web pages, file transfers, and email. UDP is connectionless, sends packets with no delivery guarantee or ordering, but is faster with lower overhead - used for DNS lookups, video calls, and online gaming where speed matters more than occasional lost packets.

TCP	UDP
Connection-oriented Reliable, ordered delivery Slower (handshake+ack) e.g. HTTP, FTP, Email	Connectionless Best-effort, unordered Faster, lower overhead e.g. DNS, video/voice calls

Q53. What is the difference between a proxy server and a reverse proxy?

Answer:

A proxy server sits between clients and the internet, forwarding client requests outward - useful for caching, content filtering, or anonymizing client identity from the destination server's perspective. A reverse proxy sits in front of one or more backend servers, forwarding client requests to them - useful for load balancing, SSL termination, and hiding backend server details from clients.

Q54. Explain congestion control mechanisms used by TCP (e.g., slow start, congestion avoidance).

Answer:

TCP congestion control prevents a sender from overwhelming the network. 'Slow start' begins with a small congestion window and doubles it each round-trip until reaching a threshold or detecting loss. 'Congestion avoidance' then increases the window more conservatively (linearly) to probe for more bandwidth without causing congestion. On packet loss, TCP reduces the window sharply (e.g. halving it) and restarts the process, adapting to network conditions.

Q55. Explain the seven layers of the OSI model and the role of each layer.

Answer:

The seven OSI layers, bottom to top: **Physical** - raw bit transmission over a medium; **Data Link** - framing and MAC addressing between directly connected nodes; **Network** - logical addressing and routing between networks (IP); **Transport** - end-to-end reliable/unreliable delivery (TCP/UDP); **Session** - managing sessions/connections between applications; **Presentation** - data formatting, encryption, compression; **Application** - the actual protocols apps use (HTTP, FTP, DNS).

OSI Model - 7 Layers



Q56. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Data Structures & Algorithms

Q57. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed $O(n \log n)$ time in best, average, and worst cases, is stable, but requires $O(n)$ extra space for merging. Quick Sort also averages $O(n \log n)$ but can degrade to $O(n^2)$ in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q58. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

Answer:

A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q59. Explain how binary search works and derive its time complexity.

Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is $O(\log n)$ - a huge improvement over linear search's $O(n)$ for large sorted datasets.

Q60. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q61. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q62. Explain the two-pointer technique with an example problem.

Answer:

The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in $O(n)$ instead of the $O(n^2)$ brute force of checking every pair.

Q63. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q64. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in $O(E \log V)$. Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q65. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually $O(1)$, but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an $O(n)$ operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to $O(1)$ amortized.

Q66. Explain topological sorting and where it is used.

Answer:

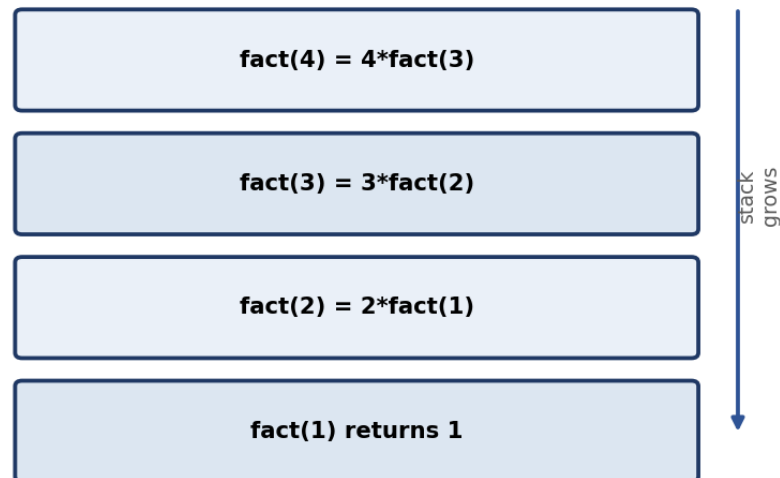
Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q67. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.

Call Stack while computing factorial(4)



Q68. What is backtracking, and how is it different from brute force?

Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Q69. How would you find the kth largest element in an unsorted array efficiently?

Answer:

One efficient approach uses a min-heap of size k: iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the kth largest, achieving $O(n \log k)$ time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average $O(n)$ time.

Q70. Explain the difference between time complexity and space complexity with examples.

Answer:

Time complexity measures how an algorithm's runtime grows as input size increases (e.g. $O(n)$ for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. $O(1)$ for an in-place swap vs $O(n)$ for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q71. How would you merge two sorted arrays/linked lists efficiently?

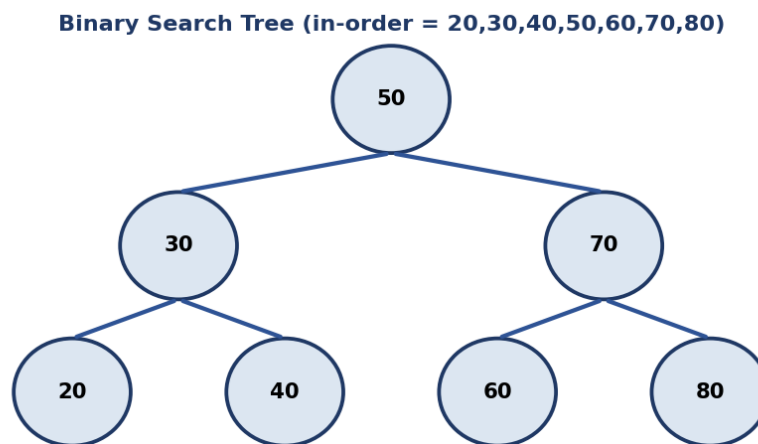
Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in $O(n + m)$ time, which is also the core merge step used inside Merge Sort.

Q72. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to $\log n$ by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing $O(\log n)$ search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to $O(n)$. Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.



Coding Problems

Q73. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return `-1/None`. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q74. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average $O(n \log n)$, worst case $O(n^2)$ with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed $O(n \log n)$ in all cases, but needs $O(n)$ auxiliary space for merging.

Sorting Algorithm Complexity Comparison

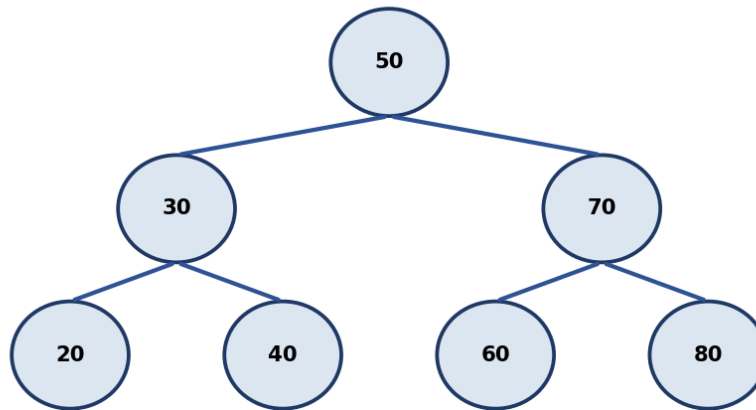
Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q75. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)

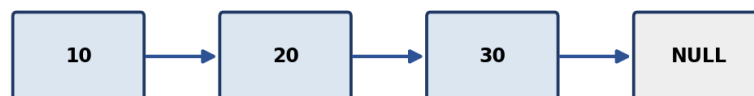


Q76. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in $O(n)$ time; iterative uses $O(1)$ extra space while recursive uses $O(n)$ call-stack space.

Singly Linked List



Q77. Write a program to merge overlapping intervals given a list of intervals.

Answer:

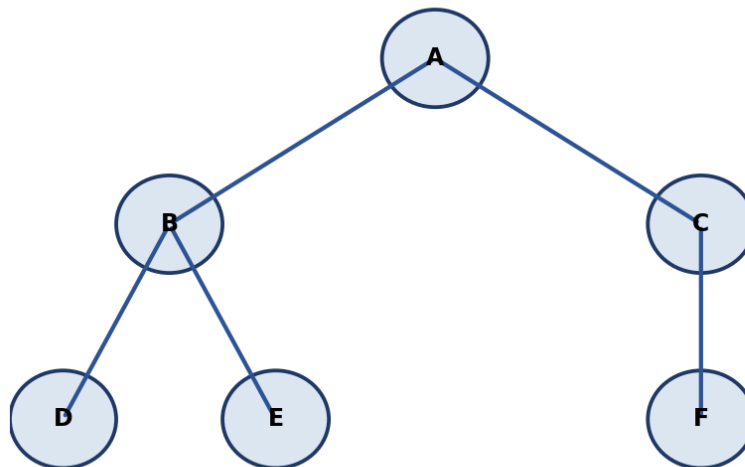
Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in $O(n \log n)$ time due to the initial sort.

Q78. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in $O(\text{rows} * \text{cols})$ time since each cell is visited a constant number of times.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F



Q79. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Q80. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity $O(n)$ where n is the total number of elements across all nesting levels.

Q81. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining $n-k$ elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q82. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table $dp[i][j]$ representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If $A[i-1] == B[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$; otherwise $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. This runs in $O(n*m)$ time and space, where n and m are the string lengths.

Q83. Write a program to check whether a given string is a palindrome.

Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in $O(n)$ time and $O(1)$ extra space (ignoring the input itself).

Q84. Write a program to find the missing number in an array containing n distinct numbers from 0 to n .

Answer:

Sum formula approach: the expected sum of numbers 0 to n is $n*(n+1)/2$; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - $O(n)$ time, $O(1)$ space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q85. Write a program to find the middle element of a linked list in a single pass.

Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in $O(n)$ time and $O(1)$ space.

Q86. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic ($\text{rear} = (\text{rear} + 1) \% \text{capacity}$) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Programming Language Fundamentals

Q87. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via `malloc/new`), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q88. What is the difference between '==' and '.equals()' when comparing objects in Java?

Answer:

'==' compares object references - whether two variables point to the exact same object in memory. '.equals()' compares the logical content/value of two objects, based on however the class has overridden it (e.g. `String`'s `.equals()` compares character sequences). Two distinct `String` objects with the same text will be '==' false but '.equals()' true, unless string interning makes them the same reference.

Q89. Explain the difference between checked and unchecked exceptions in Java.

Answer:

Checked exceptions (e.g. IOException) are checked by the compiler at compile time - a method must either handle them with try-catch or declare them with 'throws', forcing the caller to acknowledge the possibility of failure.

Unchecked exceptions (e.g. NullPointerException, extending RuntimeException) are not checked at compile time and don't require explicit handling, typically representing programming errors that could occur almost anywhere.

Q90. What is the difference between a compiler and an interpreter?

Answer:

A compiler translates the entire source program into machine code (or another target language) ahead of time, producing an executable that runs independently afterward - errors are caught before execution, and execution tends to be faster since translation work is already done. An interpreter reads and executes source code line-by-line (or statement-by-statement) at runtime without producing a standalone executable, which makes development/debugging faster but execution generally slower since translation happens repeatedly during each run.

Q91. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. `builder.setName('X').setAge(30).build()`. To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q92. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q93. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q94. What is garbage collection, and how does it help prevent memory leaks?

Answer:

Garbage collection is an automatic memory management process that identifies objects no longer reachable/referenced by the running program and reclaims their memory, so the developer doesn't have to manually free it. This helps prevent memory leaks caused by forgetting to deallocate memory, though a program can still 'leak' memory logically if it keeps unnecessary references alive (e.g. objects sitting in a growing cache that's never cleared).

Q95. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q96. Explain the difference between pass-by-value and pass-by-reference with an example.

Answer:

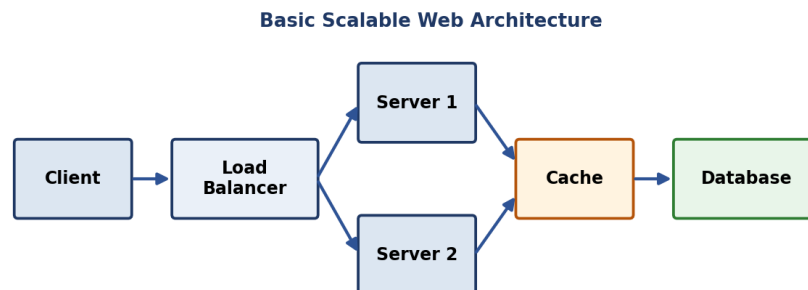
Pass-by-value copies the actual value of a variable into the function's parameter, so changes inside the function don't affect the original variable. Pass-by-reference passes a reference to the original variable's memory location, so changes inside the function do affect the original. Example: in a language with true pass-by-reference, `swap(a, b)` can actually swap the caller's variables; with pass-by-value, it would only swap local copies, leaving the caller's variables unchanged.

Web & Cloud Fundamentals

Q97. What is the purpose of an API Gateway in a microservices-based application?

Answer:

An API Gateway sits in front of a microservices architecture as the single entry point for client requests, routing each request to the appropriate backend service. It commonly also handles cross-cutting concerns centrally - authentication/authorization, rate limiting, request/response transformation, and logging - so individual microservices don't each need to reimplement that logic.



Q98. Explain the concept of statelessness in REST APIs and why it matters for scalability.

Answer:

Statelessness means each request from a client to a server must contain all the information the server needs to process it, with no reliance on server-side session state stored between requests. This matters for scalability because any server instance can handle any request without needing shared session state, making it trivial to add more server instances behind a load balancer without worrying about which server 'remembers' a given client.

Q99. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q100. Explain how a CDN improves website performance for geographically distributed users.

Answer:

A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

Note: 15 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.